

Code Explanation

SCD Type 2 Update Logic Explanation

The provided code is a Python function named `update\_scd\_type2\_table` that updates a Slowly Changing Dimension (SCD) Type 2 table called `ordersummary` when customer data changes. This function is designed to be used outside of Delta Live Tables (DLT) pipelines.

Overview

The function takes in a SparkSession, catalog name, and schema name as arguments. It reads the current customer and order data, generates new records based on this data, and then updates the `ordersummary` table according to the SCD Type 2 logic. This involves updating existing records where customer details have changed and inserting new records for the updated customers.

Step 1: Read Current Customer and Order Data

This step reads the current customer and order data from the `customer\_silver` and `order\_silver` tables, respectively.

```
customer_df = spark.table(f"{catalog}.{schema}.customer_silver")
order_df = spark.table(f"{catalog}.{schema}.order_silver")
```

Step 2: Read Current ordersummary Table

This step reads the current `ordersummary` table into a DeltaTable object and converts it to a DataFrame.

```
ordersummary_table = DeltaTable.forName(spark, f"{catalog}.{schema}.ordersummary")
ordersummary_df = ordersummary_table.toDF()
```

Step 3: Generate New Records Based on Current Customer and Order Data

This step joins the customer and order DataFrames on the `CustId` column, selects the required columns, and adds new columns for `IsActive`, `StartDate`, and `EndDate`.

```
new_records = (
    order_df
    .join(
        customer_df,
        on="CustId",
        how="inner"
    )
    .select(
        "CustId", "Name", "EmailId", "Region", "OrderId",
        "ItemName", "PricePerUnit", "Qty", "Date"
    )
    .withColumn("IsActive", F.lit(True))
    .withColumn("StartDate", F.current_timestamp())
    .withColumn("EndDate", F.lit(None).cast(TimestampType()))
)
```

Step 4: Find Records That Need to be Updated

This step defines a join condition to identify records in the `ordersummary` table that need to be updated due to changes in customer details.

```
join_condition = (
    (ordersummary_df["CustId"] == new_records["CustId"]) &
    (ordersummary_df["OrderId"] == new_records["OrderId"]) &
    (
        (ordersummary_df["Name"] != new_records["Name"]) |
        (ordersummary_df["EmailId"] != new_records["EmailId"]) |
        (ordersummary_df["Region"] != new_records["Region"])
    ) &
    (ordersummary_df["IsActive"] == True)
)
```

Step 5: Perform the SCD Type 2 Update

This step uses the `merge` function to update the `ordersummary` table. It sets `IsActive` to `False` and updates the `EndDate` to the current timestamp for the records that match the join condition.

```
ordersummary_table.alias("target").merge(
    new_records.alias("source"),
    join_condition
).whenMatchedUpdate(
    set={
        "IsActive": F.lit(False),
        "EndDate": F.current_timestamp()
    }
).execute()
```

Step 6: Insert New Records for the Updated Customers

This step identifies the new records that need to be inserted into the `ordersummary` table and writes them to the table in append mode.

```
matched_records = (
    new_records.join(
        ordersummary_df.filter(F.col("IsActive") == False),
        on=["CustId", "OrderId"],
        how="inner"
    )
    .select(new_records["*"])
)

if matched_records.count() > 0:
    matched_records.write.format("delta").mode("append").saveAsTable(
        f"{catalog}.{schema}.ordersummary")
```